

Scientific Computation

Prof. Matthew Giamou
Sept. 18, 2025

Lecture 8 - Linear Systems

Lecture Outline

- Linear systems refresher

- Gaussian elimination
- Backward substitution
- Complexity analysis

Linear Systems

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^n$, solve

$$Ax = b,$$

where $x \in \mathbb{R}^m$.

Each $a_i^\top \in \mathbb{R}^n$ + b_i
form a single equation.
for a total of m linear
equations.

$$\begin{bmatrix} a_1^\top \\ a_2^\top \\ \vdots \\ a_m^\top \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Second Interpretation

Given a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^n$, solve

Each $a_i \in \mathbb{R}^m$ is a vector, x_i is a coordinate, we sum weighted a_i 's to equal b .

$$Ax = b,$$

where $x \in \mathbb{R}^m$.

$$\begin{bmatrix} | & | & \dots & | \\ a_1 & a_2 & \dots & a_n \\ | & | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

Solvability

A square system has a unique solution if it is **nonsingular**

The following are equivalent:

- A is non singular
- $\det(A) \neq 0$
- columns/rows are linear independent
- There exists A^{-1} s.t. $A^{-1}A = AA^{-1} = I$
- Eigenvalues of A are all non zero

$$(A - \lambda I)_{\lambda=0} \Leftrightarrow \ker(A)$$

5

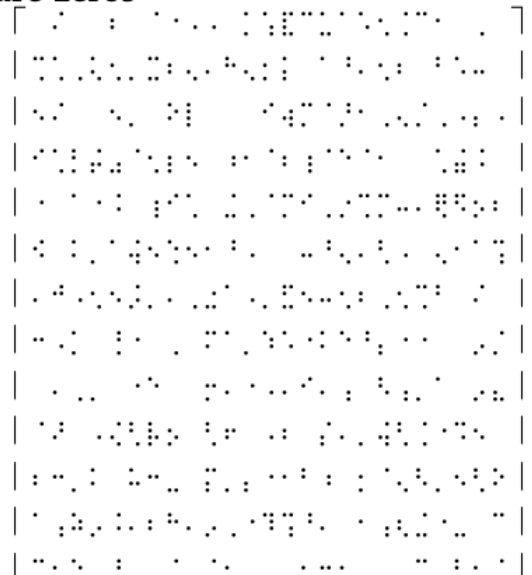
Sparse Systems

In general, we will assume that matrices are **dense** (i.e., they have a small number of nonzero entries)

In a **sparse** system, most of the elements are zeros

```
using SparseArrays
p = 0.2
A = sprand(50, 50, p)
```

Specialized algorithms exist for working with very large but sparse matrices!



6

Solution Methods

There are two broad classes of methods for solving linear systems:

- **Direct methods:** based on Gaussian elimination (today), can be specialized from particular matrix structure

- **Iterative methods:** iteratively improve an approximation, needed for very large systems

This course focuses on **direct methods**, but we'll learn a bit about iterative methods too

7

Lecture Outline

- Linear systems refresher
- **Gaussian elimination**
- Backward substitution
- Complexity analysis

Gaussian Elimination by Hand

GAUSSIAN ELIMINATION BY HAND

Example 1: Let's work through a small example by hand.

$$Ax = \begin{bmatrix} 1 & -1 & 3 \\ 1 & 1 & 0 \\ 3 & -2 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 11 \\ 3 \\ 3 \end{bmatrix} = b$$

"Clear" the first column: Multiply the first row by 1 and subtract from the second row; multiply the first row by 3 and subtract from the third row.

$$A|b = \left[\begin{array}{ccc|c} 1 & -1 & 3 & 11 \\ 1 & 1 & 0 & 3 \\ 3 & -2 & 1 & 3 \end{array} \right] \begin{array}{l} \downarrow \times 1 \\ \downarrow \times 3 \end{array}$$

$$= \left[\begin{array}{ccc|c} 1 & -1 & 3 & 11 \\ 0 & 2 & -3 & -8 \\ 0 & 1 & -8 & -30 \end{array} \right] \begin{array}{l} \times \frac{1}{2} \\ \downarrow \end{array}$$

9

Gaussian Elimination Cont.

Example 1 cont.: Multiply second row by 0.5 and subtract from third row

$$A|b = \left[\begin{array}{ccc|c} 1 & -1 & 3 & 11 \\ 0 & 2 & -3 & -8 \\ 0 & 1 & -8 & -30 \end{array} \right]$$

$$\left[\begin{array}{ccc|c} 1 & -1 & 3 & 11 \\ 0 & 2 & -3 & -8 \\ 0 & 0 & -6.5 & -2 \end{array} \right]$$

Any 0's rows on the bottom

This is called Gaussian elimination or forward elimination, it puts the matrix in row echelon form

10

Gaussian Elimination Code

```
#####
| Perform Gaussian elimination in-place
#####
function gaussian_elimination!(A, b)
    n = size(A, 1)
    for k in 1:n-1 # For each row except the last
        for i in k+1:n # Remove a multiple of the kth row
            m_ik = A[i,k]/A[k,k] # The multiplier will clear the kth column
            # Update both A and b
            for j in k:n
                A[i,j] = A[i,j] - m_ik*A[k,j]
            end
            b[i] = b[i] - m_ik*b[k]
        end
    end
end
```

This is also called **forward** elimination - note that this code is **unsafe and incomplete!** Why?

11

Lecture Outline

- Linear systems refresher
- Gaussian elimination
- **Backward substitution**
- Complexity analysis

Backward Substitution

Example 2: Let's finish solving the system from Example 1, which we now have in row echelon form:

$$\tilde{A}x = \begin{bmatrix} 1 & -1 & 3 \\ 0 & 2 & -3 \\ 0 & 0 & -6.5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 11 \\ -8 \\ -26 \end{bmatrix} = \tilde{b}$$

$(MA)_x = Mb$
 $\tilde{A} = MA$
 $\tilde{b} = Mb$
 $\tilde{A}x = \tilde{b}$

$$x_3 = \frac{b_3}{a_{33}} = \frac{-26}{-6.5} = 4$$

$$x_2 = \frac{b_2 - a_{23}x_3}{a_{22}} = \frac{-8 - (-3) \cdot 4}{2}$$

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}} = \frac{11 - (-1) \cdot 2 - 3 \cdot 4}{1} = 1$$

This is called **backward substitution**, and it exploits row echelon form

Lecture Outline

- Linear systems refresher
- Gaussian elimination
- Backward substitution
- Complexity analysis

Gaussian Elimination: Counting Operations

Let's count the number of operations (ignore b for simplicity).

Innermost loop:

```
for j in k:n
    A[i,j] = A[i,j] - m_ik*A[k,j]
end
```

$n-k+1$ iterations



↳ ignore because we can zero the leading element

It does $n-k+1$ multiplications
 $n-k+1$ subtractions

note: $A[i,k] = 0$, no calculation needed

$\Rightarrow 2(n-k)$ flops

15

Gaussian Elimination: Counting Operations Cont.

Let's count the number of operations (ignore b for simplicity).

Next nested loop (moving out):

```
for i in k+1:n # Remove a multiple of
    m_ik = A[i,k]/A[k,k] # The multipl
    # Update both A and b
    for j in k:n
        A[i,j] = A[i,j] - m_ik*A[k,j]
    end
```

16

Gaussian Elimination: Counting Operations Cont.

Let's count the number of operations (ignore b for simplicity).

Finally, the outermost loop:

```
for k = 1:n-1
```

```
for k in 1:n-1 # For each row except the
    for i in k+1:n # Remove a multiple of
        m_ik = A[i,k]/A[k,k] # The multip
        # Update both A and b
        for j in k:n
            A[i,j] = A[i,j] - m_ik*A[k,j]
        end
```

17

Gaussian Elimination: Counting Operations Cont.

We can get the complexity after doing some algebra:

18

Gaussian Elimination: Counting Operations Cont.

The next term is simpler: